

第 13 章 | TensorFlow 與 Keras：從張量到深度學習

學生版公開講義 | 請搭配本課程 Colab，依照段落逐步操作與自我檢查。

來源說明：課程參考《Python Machine Learning, 3rd Edition》；原始範例程式碼採 MIT License。本檔由恩恩統計家教整理為學生版學習摘要，不含原書 PDF 或掃描內容。

1. 本章學習路線與導引

前一章以 NumPy 拆解神經網路的計算流程；本章改用 TensorFlow 與 Keras 處理參數較多、適合平行運算的模型，並建立可重複執行的訓練流程。

本章介紹 TensorFlow 與 Keras。你將使用張量運算、自動微分與 GPU 加速，把前一章的神經網路觀念改寫成可維護的框架實作。掌握框架能幫助你把模型觀念轉化為可維護的工程實作。

學習目標

掌握張量（Tensor）的核心概念：理解 TensorFlow 的唯一資料格式及其與 NumPy 的相容特性。

建立高效資料管線（`tf.data.Dataset`）：學習在記憶體受限下，如何實現邊抽邊餵的高效機制。

熟練 Keras 高階 API：使用一致的介面建構、編譯與訓練模型。

辨析激勵函數的物理特性：從數學與梯度導向選擇適合深層網路的激勵函數（如 ReLU）。

課前心理建設

TensorFlow 以張量為資料單位，並提供自動微分、計算圖與硬體加速。你仍需理解數學原理，但不必手動撰寫每一個梯度與記憶體操作。

使用框架前，先比較 NumPy 手刻流程與 TensorFlow 在計算及資料管線上的差異。

2. 核心觀念：為什麼需要 TensorFlow？

在處理看似簡單的 MNIST 手寫辨識任務時，若網路僅包含 100 個隱藏層單元，我們就必須最佳化將近 80,000 個權重參數（精確計算為 79,510 個）。若影像解析度提高或增加網路深度，參數數量會快速增加，這對傳統硬體構成了巨大挑戰。

傳統 CPU 受限於 Python 的全域解釋器鎖（GIL），預設僅能單核執行，且核心數極少。相比之下，GPU 是為了矩陣運算而生的工作馬。

效能瓶頸評估：CPU vs. GPU

以下根據來源資料中 2019 年的典型硬體規格對比：

規格維度	Intel Core i9-9960X (CPU)	NVIDIA RTX 2080 Ti (GPU)	效能差異
核心數	16 Cores	4,352 Cores	GPU 核心數是 CPU 的 272 倍

規格維度	Intel Core i9-9960X (CPU)	NVIDIA RTX 2080 Ti (GPU)	效能差異
浮點運算能力	約 1.3 TFLOPS	約 13.4 TFLOPS	GPU 約高出 10 倍以上
性價比 (2019 資料)	100% 價格	約 65% 價格	在適合平行運算的工作負載下，GPU 通常具有較高效能

白話觀念說明：手寫 NumPy 與 TensorFlow 的分工

手寫 NumPy 網路適合觀察每一步計算；TensorFlow 則負責自動微分、運算排程與硬體加速。實際加速幅度取決於模型、資料量與執行環境。

接著先理解 TensorFlow 的基本資料單位：張量。

3. 重要術語：張量 (Tensor) 的多維世界

張量 (Tensor) 是 TensorFlow 的唯一資料格式。雖然名稱聽起來艱澀，但請直觀地將其視為「能在 GPU 上運算的 NumPy 陣列」。TensorFlow 內部以 NumPy 陣列儲存數值，並提供符號控制碼 (Symbolic Handle) 進行運算。

術語對照表：張量階級 (Rank)

「階 (Rank)」代表了資料的維度數量：

階級 (Rank)	中文術語	物理概念描述	應用範例
Rank 0	純量 (Scalar)	單一個點	一個數值 (如 Loss 值)
Rank 1	向量 (Vector)	一排點 (線)	單一樣本的特徵向量
Rank 2	矩陣 (Matrix)	平面表格	批次處理的二維資料表
Rank 3	三維張量 (3D Tensor)	立體方塊	彩色圖片 (高 x 寬 x RGB 通道)

視覺化概念解說

Rank 0：想像一個孤立的點。

Rank 1：點排成線，具有方向。

Rank 2：線展開成面，形成了寬與高的表格。

Rank 3：平面堆疊成體，像是一疊相片或一個三維魔術方塊。

觀念澄清：Rank vs. Shape

Rank (階)：告訴你有幾個「維度方向」 (例如 Rank 2 有橫向與縱向)。

Shape (形狀)：告訴你每個方向具體「有多長」 (例如 Shape (3, 5) 代表有 3 橫行 5 縱列)。

4. 程式碼流程導讀：張量操作與資料管線

當資料量大到無法一次放入記憶體 (如 100GB 影像) 時，你必須建立高效的資料輸入管線 (Data Pipeline)，這正是 `tf.data` API 的實務價值所在。

基礎操作指令集

`tf.convert_to_tensor()`：資料格式轉換。

`tf.cast()` 與 `tf.reshape()`：形狀與型態變換（注意：在訓練迴圈中頻繁轉換 NumPy 與 Tensor 會造成 I/O 延遲瓶頸）。

`tf.linalg.matmul()`：GPU 加速的矩陣乘法。

資料順序與 Buffer Size

在建立 `tf.data.Dataset` 時，函數串接的順序與參數設定會決定模型能否學到東西：

正確順序：`.shuffle(buffer_size).batch()` 這會先在暫存區進行打亂，再打包。`buffer_size` 的設定極其關鍵：它代表一個「暫存箱」的容量。若 `buffer_size` 太小（例如 100GB 資料只設 10），洗牌會極不均勻，導致模型學習能力下降。實作建議：對於中小型資料集，`buffer_size` 應等於樣本總量。

錯誤順序：`.batch().shuffle()` 這會先打包資料，然後只「交換箱子」的順序。箱子內部的資料組合永遠固定，模型將無法學到真正的隨機特徵。

資料輸送帶比喻

請想像 `tf.data.Dataset` 是一條工廠輸送帶：

Map：輸送帶上的機器對原料進行前處理（如縮放圖片）。

Shuffle：原料進入一個暫存箱（Buffer）進行隨機抽籤。

Batch：處理好的原料每 32 個裝成一箱，送到神經網路門口。

5. Keras API：樂高式的模型建立

`tf.keras` 是官方推薦的高階 API。它的核心哲學是簡潔與模組化。

樂高積木比喻

模型（Sequential）：是一個底座。

層（Layer）：是不同形狀的積木，定義了運算規則。

參數（Weights）：是積木內部的「電子電路」。訓練過程就是在不改變積木形狀的情況下，微調電路的強弱。

模型三部曲

建構：堆疊 `Dense` 層並設定 `input_shape`。

技術提示：若未設定 `input_shape`，TensorFlow 會啟動「延遲建立（Late variable creation）」機制，直到第一次餵資料前，模型都不會真正建立參數，這也是為什麼有時 `summary()` 會顯示問號或報錯的原因。

編譯 (Compile)：設定 Adam（具備自動調整學習率優點）、Loss（如 `sparse_categorical_crossentropy`）與指標。

擬合 (Fit)：接上資料輸送帶，設定 Epochs 進行全自動化訓練。

實作建議：`model.summary()`

這不只是體檢工具，它是進入大型工業專案時，與團隊對接架構的唯一公約數。在訓練前檢視它，能確保層次連接與參數量符合預期。

6. 深度學習的法官：激勵函數 (Activation Functions)

激勵函數為網路引入「非線性」，否則多層網路本質上仍只是單一層線性變換。

主流激勵函數評估

函數名稱	特性與數學優勢	缺陷分析
Sigmoid / Tanh	將數值壓縮在 (0, 1) 或 (-1, 1)	梯度消失：當輸入極大/小時，曲線平坦，斜率趨近 0，導致誤差無法回傳。
ReLU	$f(z)=\max(0,z)$	現代標配：當 $z>0$ 時，導數 (Gradient) 恆為 1。這確保了誤差能無損地傳回深層網路，解決了梯度消失。

輸出層專項指導：對症下藥

二元分類：使用 Sigmoid。

多元分類：使用 Softmax（計算出所有類別機率且總和為 1），這能提供具解釋性的機率分佈。

7. Colab 實機操作、常見錯誤與排除

Colab 加速步驟

開啟 GPU：執行階段 -> 變更執行階段類型 -> 硬體加速器選擇 GPU。

驗證環境：`import tensorflow as tf; print(tf.__version__)`。

常見錯誤 (Troubleshooting) 表格

錯誤現象	核心原因	排除方案
訓練 Loss 幾乎不動	發生「梯度消失」	隱藏層更換為 ReLU。
訓練速度非常慢	I/O Bottleneck	避免在 Loop 內進行 NumPy 與 Tensor 頻繁切換。
summary() 看不到資訊	延遲建立 (Late variable creation)	在第一層添加 input_shape。
資料組合僵化	順序陷阱或 Buffer 太小	確保 .shuffle() 在 .batch() 之前，且調大 buffer_size。

動手練習與自我檢查

實機練習：使用 `tfds.load('iris')` 載入資料。建立一個 `Sequential` 模型，包含兩層隱藏層（16 與 8 個神經元，ReLU），並對其進行訓練。

參數量計算公式：請手動驗證 `summary()` 中的參數。公式： $(nin+1) \times nout$ （其中 1 是 Bias 項）。

模型保存筆記：使用 `model.save('iris-classifier.h5')`。請注意，.h5 格式會同時保存模型架構與所有訓練好的權重參數。

自我檢查清單

- ☐ 我是否理解了 Rank 與 Shape 的差異？
- ☐ 我能否解釋為何 ReLU 在 $z > 0$ 時導數為 1 的數學重要性？
- ☐ 我是否能在 Colab 裡成功開啟 GPU 加速？

本章總結：本章介紹張量操作、Keras 模型與資料管線。請用不同批次大小重跑範例，觀察訓練時間與結果差異；下一章將進一步拆解權重更新與客製化模型。